Lecture 28.11: Post-Pruning Implementation (Cost-Complexity ccp_alpha)

📌 Topic Overview

In this lecture, we study **Post-Pruning** using **Minimal Cost-Complexity Pruning** (`ccp_alpha`). We let the tree grow completely, compute the pruning path with `cost_complexity_pruning_path()` , and find the optimal alpha parameter that maximizes generalization.

📐 Mathematical Formulation: Cost-Complexity Cost Function

Cost-complexity pruning defines a cost function that balances classification error with tree size:

$$R_\alpha(T) = R(T) + \alpha|T|$$

Where:

- $R(T)$ = Total training misclassification error of tree T.
- $|T|$ = Number of terminal leaf nodes in tree T (a measure of tree complexity).
- $\alpha \geq 0$ = Complexity penalty parameter (`ccp_alpha`).
- When $\alpha = 0$: Standard unpruned tree (pure leaves, high complexity).
- As $\alpha \to \infty$: Penalty dominates; the tree gets pruned all the way back to a single root node.

🛠️ Step-by-Step Implementation from Lecture

1. Extract the Pruning Path

Scikit-Learn provides `cost_complexity_pruning_path()` , which returns the exact candidate alpha values where the tree structure changes:

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

def optimize_pruning_ccp_alpha(X_train, y_train, X_test, y_test):
    """
```

```
    Finds the optimal cost-complexity pruning alpha (ccp_alpha)
    by calculating effective alphas and evaluating accuracy on test data.

    Parameters:
        X_train, y_train: Training features and labels
        X_test, y_test:   Test/Validation features and labels

    Returns:
        best_alpha (float): Optimal ccp_alpha value
        best_score (float): Highest test accuracy achieved
        best_model (DecisionTreeClassifier): Best pruned tree
    """
    # 1. Pruning path: Get candidate effective alphas from unconstrained tree
    base_tree = DecisionTreeClassifier(random_state=42)
    path = base_tree.cost_complexity_pruning_path(X_train, y_train)
    ccp_alphas = path.ccp_alphas

    # 2. Iterate through all candidate alphas and evaluate test score
    best_alpha = 0.0
    best_score = 0.0
    best_model = None

    for alpha in ccp_alphas:
        clf = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
        clf.fit(X_train, y_train)
        score = clf.score(X_test, y_test)

        # Track the best alpha that maximizes generalization
        if score > best_score:
            best_score = score
            best_alpha = alpha
            best_model = clf

    # 3. Return best pruned model and metrics
    return best_alpha, best_score, best_model

# --- Function Call & Testing ---
# best_alpha, best_score, pruned_tree = optimize_pruning_ccp_alpha(X_train, y_trai
# print(f"Best ccp_alpha:       {best_alpha:.5f}")
# print(f"Optimized Test Score: {best_score:.4f}")
```

2. Train a Model for Each Candidate Alpha

In the video, the instructor loops through `ccp_alphas`:

```
# Train a model for each candidate alpha
trees = []
for alpha in ccp_alphas:
    model = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
```

```
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    trees.append((model, acc, alpha))
```

3. Find the Best Alpha

We select the alpha that gives the maximum test accuracy:

```
best_tree, best_acc, best_alpha = max(trees, key=lambda x: x[1])
print(f"Optimal ccp_alpha: {best_alpha:.5f}")
print(f"Best Test Accuracy: {best_acc:.4f}")
```

💻 Python Implementation

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

def find_optimal_ccp_alpha(X_train, y_train, X_test, y_test):
    """
    Finds the optimal cost-complexity pruning alpha (ccp_alpha)
    by evaluating candidate alphas on test data.

    Parameters:
        X_train, y_train: Training data and labels
        X_test, y_test:   Validation/Test data and labels

    Returns:
        best_alpha (float): The optimal alpha parameter
        best_score (float): Highest test accuracy achieved
        best_model (DecisionTreeClassifier): Fitted pruned tree
    """
    # 1. Pruning path: Extract candidate effective alphas
    base_tree = DecisionTreeClassifier(random_state=42)
    path = base_tree.cost_complexity_pruning_path(X_train, y_train)
    ccp_alphas = path.ccp_alphas

    # 2. Iterate through all alphas and evaluate
    best_alpha = 0.0
    best_score = 0.0
    best_model = None

    for alpha in ccp_alphas:
        clf = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
        clf.fit(X_train, y_train)
        score = clf.score(X_test, y_test)

        # Track the alpha with highest test accuracy
```

```
        if score > best_score:
            best_score = score
            best_alpha = alpha
            best_model = clf

    # 3. Return results
    return best_alpha, best_score, best_model

# --- Function Call & Testing ---
# best_alpha, best_score, pruned_tree = find_optimal_ccp_alpha(X_train, y_train, X
# print(f"Optimal ccp_alpha:   {best_alpha:.5f}")
# print(f"Best Test Accuracy: {best_score:.4f}")
```

🔑 Key Takeaways

1. Post-pruning lets the tree grow fully, then trims weakest leaves bottom-up.
2. `cost_complexity_pruning_path()` provides exact candidate `ccp_alpha` thresholds.
3. Increasing `ccp_alpha` simplifies the tree by penalizing the number of leaf nodes.